

1. What This Project Is

A backend system (an API — a program other programs or interfaces talk to) that lets a user track personal finances: income, expenses, spending categories, monthly budgets, and a calculated summary report. There is no visual website in front of it (yet) — it's tested through an auto-generated web page called Swagger UI, and could later be connected to a real frontend (HTML/CSS/JS, a mobile app, anything that can send HTTP requests).

Why build this project specifically? It touches almost every core skill a backend developer needs on day one of a real job: authentication, database design, input validation, relationships between data, and calculated reporting — all in one small, understandable system.

2. Tech Stack — What Each Piece Does and Why It Was Chosen

Tool	Role	Why this one
Python	The programming language everything is written in	Widely used for backend, readable syntax
FastAPI	Web framework — receives HTTP requests, routes them to functions	Modern, fast, auto-generates interactive docs, has built-in data validation
Pydantic	Data validation — defines the exact shape of valid input/output	Comes bundled with FastAPI, works via plain Python classes
SQLAlchemy	ORM (Object-Relational Mapper) — lets Python classes represent database tables	Avoids writing raw SQL by hand; database-agnostic (same code works with MySQL, PostgreSQL, etc.)
MySQL	The actual database — stores data permanently on disk	Widely taught/used, free, reliable
JWT (JSON Web Tokens)	Authentication — proves who's making a request without a server-side session	Stateless, industry-standard for APIs
passlib + bcrypt	Password hashing	Never store real passwords — store an irreversible scrambled version instead
Uvicorn	ASGI server — the program that actually runs the FastAPI app and listens on a port	Required to execute the app; FastAPI code alone can't run itself
Git / GitHub	Version control and code hosting	Industry standard for tracking changes and sharing code

3. Database Design

Four main tables, plus one supporting table:

```
users
└─ categories (belongs to a user)
```

- └ transactions (belongs to a user, optionally tagged with a category)
- └ budgets (belongs to a user AND a category, for a specific month/year)

token_blacklist (standalone – tracks logged-out tokens)

Table: users

Column	Type	Notes
id	integer	primary key
email	string	unique
hashed_password	string	never the real password
full_name	string	optional
created_at	datetime	

Table: categories

Column	Type	Notes
id	integer	primary key
name	string	e.g. “Food”
owner_id	integer	foreign key → users.id

(name + owner_id together must be unique — you can’t have two “Food” categories for the same user)

Table: transactions

Column	Type	Notes
id	integer	primary key
type	enum	“income” or “expense”
amount	float	must be > 0
description	string	optional
date	datetime	defaults to now
owner_id	integer	foreign key → users.id
category_id	integer	foreign key → categories.id (optional)

Table: budgets

Column	Type	Notes
id	integer	primary key
amount	float	the budget limit
month, year	integer	which month this budget applies to
owner_id	integer	foreign key → users.id
category_id	integer	foreign key → categories.id

(owner_id + category_id + month + year together must be unique — setting a budget twice for the same month **updates** it instead of duplicating)

Table: token_blacklist

Column	Type	Notes
--------	------	-------

id	integer	primary key
token	string	a JWT that's been logged out
created_at	datetime	

4. Folder Structure

```

expense_api/
├── app/
│   ├── main.py           # Creates the FastAPI app, wires everything together
│   ├── config.py         # Reads settings (DB connection, secret key) from .env
│   ├── database.py       # Sets up the SQLAlchemy connection to MySQL
│   ├── models.py         # The 5 tables, defined as Python classes
│   ├── schemas.py        # Defines valid input/output shapes for the API
│   ├── security.py       # Password hashing + JWT creation/verification
│   └── dependencies.py   # Reusable logic: "get a DB session", "who is this user"
├── routers/
│   ├── auth.py           # /auth/register, /auth/login, /auth/logout
│   ├── categories.py     # /categories
│   ├── transactions.py   # /transactions
│   ├── budgets.py       # /budgets
│   └── reports.py        # /reports/monthly
├── requirements.txt       # List of Python packages this project needs
├── .env                  # Your actual secrets (never shared/uploaded)
└── .env.example          # A template showing what .env should contain

```

Each file has exactly one job. This separation (called “separation of concerns”) is why real projects are split into many small files instead of one giant script — each piece can be understood, tested, and changed on its own.

5. How a Single Request Actually Flows Through the Code

Example: a logged-in user calls `POST /transactions` to add an expense.

Step 1 — FastAPI receives the request. `main.py` has already registered all the routers, so FastAPI knows this URL belongs to `routers/transactions.py`.

Step 2 — Dependencies run first. Before your actual function executes, FastAPI resolves everything the function asked for via `Depends(...)`: `- get_db()` (from `dependencies.py`) opens a database session for this one request. `- get_current_user()` reads the `Authorization: Bearer <token>` header, decodes the JWT using `security.py`, checks it isn't in `token_blacklist`, and looks up the matching row in the `users` table.

If the token is missing, invalid, or blacklisted, the request stops right here with a `401 Unauthorized` — your function never even runs.

Step 3 — Pydantic validates the incoming JSON. The request body is checked against `TransactionCreate` in `schemas.py`. If `amount` is missing, negative, or the wrong type, the request stops with a `422 Validation Error` before your logic runs.

Step 4 — Your route function runs. Inside `routers/transactions.py`, the function: 1. Optionally checks the given `category_id` actually belongs to this user (so users can't attach transactions to someone else's category). 2. Creates a `Transaction` object (from `models.py`) with the validated data. 3. Adds it to the database session and commits — this is the moment SQLAlchemy generates and runs the actual SQL `INSERT` statement.

Step 5 — The response is shaped and sent back. The new `Transaction` object is converted into JSON using the `TransactionOut` schema, and FastAPI sends it back with a `201 Created` status.

This exact five-step pattern (auth check → validate input → run logic → touch the database → shape the output) repeats for every single endpoint in the project.

6. Authentication, Explained Fully

1. **Register:** the plaintext password is run through `bcrypt` (via `passlib`), which turns it into an irreversible scrambled string. Only this scrambled string is stored — never the real password.
 2. **Login:** the user's submitted password is scrambled the same way and compared to the stored hash. If they match, the server creates a JWT — a signed string containing the user's ID and an expiry time — signed using a secret key only the server knows.
 3. **Using the API:** every subsequent request includes this token in its header. The server verifies the signature (proving it wasn't tampered with) and reads the user ID from inside it — no database lookup needed just to know *who* is asking, which is what makes JWT “stateless.”
 4. **Logout:** since a valid JWT normally can't be “cancelled” early (it's just a signed piece of text), this project adds a `token_blacklist` table — a request is rejected if its token appears there, even if the token hasn't technically expired yet.
-

7. Why This Design Was Chosen (Talking Points)

- **Per-user data isolation:** every single database query filters by `owner_id`, sourced from the verified JWT — never from user input. This is what stops User A from ever seeing or editing User B's data.
 - **Upsert budgets:** rather than allowing duplicate budget rows for the same category/month, setting a budget again updates the existing one — simpler for the user, cleaner data.
 - **Optional category on transactions:** income doesn't always need a category, so `category_id` is optional — but once given, it's validated to belong to the same user.
 - **Separation of schemas vs. models:** `models.py` (database shape) and `schemas.py` (API input/output shape) are deliberately different files, even though they overlap — this lets the API hide internal fields (like `hashed_password`) and validate input independently of how it's stored.
-

End of Project Deep Dive.